案例一 · 用 Docker 测试静态网站

第 14 讲

目标

把一套静态站在 Docker 里"测明白", 不留痕

- 测试诉求: 隔离、可复现、测完即弃
- Sample 初始 Dockerfile: `nginx:alpine` + COPY
- 两种宿主: Nginx 镜像 vs **Tomcat 镜像**
- 从镜像建容器, 改网站再验证

一、为什么用 Docker 测

> 试错不弄脏本机

测试怕两件事: 污染本机、环境不一

- 装错依赖、版本冲突, 后患无穷
- "在我这是好的" —— 团队每台机器都不一

> Docker 把"网站+环境"打包: 走到哪都一样, 测完即弃

案例三步走

```
① 写 Sample 初始 Dockerfile
② 构建镜像，跑起来并验证(curl)
③ 从镜像建容器，改网站，再验证
```



> 每步都用前面学过的命令, 是本课程"综合题"

二、Sample 的初始 Dockerfile

> 静态站, Nginx 最省事

两行搞定: 拉起 Nginx, 放进网站文件

```
# site/index.html 已在本地
FROM nginx:alpine
COPY site/ /usr/share/nginx/html/
EXPOSE 80
```

`COPY site/` 把整个静态站放进 Nginx 默认根目录

三、构建并测试 Sample 网站

> build → run → curl

一条龙验证

```
docker build -t sample-site .
docker run -d --name sample -p 8080:80 sample-site
curl localhost:8080        # 返回 Sample 页面 ✓
docker logs sample         # 看访问日志
docker exec -it sample ls /usr/share/nginx/html   # 文件在
```



> 能返回页面 + 日志正常 = 测试通过

四、Sample + Tomcat 镜像

> 换了块"宿主"

网站作为 webapp 放进 Tomcat

```
FROM tomcat:10
COPY site/ /usr/local/tomcat/webapps/sample/
```

```
docker build -t sample-tomcat .
docker run -d --name sample-tc -p 8080:8080 sample-tomcat
curl localhost:8080/sample/    # 访问到 Sample
```

> webapps 下的目录名 = 访问路径(/sample/)

五、从镜像建容器, 改网站

> 修改有三条路

改网站: 从 bind mount 到 exec

```
# ① bind mount(推荐): 改宿主文件, 挂进去即时生效
echo '<h1>v2</h1>' > site/index.html
docker run -d --name sample -p 8080:80 \
  -v "$PWD/site":/usr/share/nginx/html sample-site
curl localhost:8080        # v2

# ② exec 直改: 进容器临时改(删容器就丢)
docker exec -it sample sh -c \
  'echo hi > /usr/share/nginx/html/index.html'
```



> 长期修改用 mount / 重建; exec 只配临时验证

三路对比

方式	持久	适用
改后 `build` 重建	✓	交付/上线
bind mount	✓	开发调试
`exec` 直改	✕ 删即丢	快速验证

结论: 要持久用 mount 或 build, exec 只在应急

六、完整实战

静态站测试收官

> 构建→测试→改→再测, 一次走通

一次走到底

```
mkdir -p site && echo '<h1>Sample v1</h1>' > site/index.html

echo 'FROM nginx:alpine' > Dockerfile
cat >> Dockerfile <<'EOF'
COPY site/ /usr/share/nginx/html/
EOF
docker build -t sample-site .
docker run -d --name sample -p 8080:80 sample-site
curl localhost:8080        # Sample v1

echo '<h1>Sample v2</h1>' > site/index.html
docker rm -f sample
docker run -d --name sample -p 8080:80 \
  -v "$PWD/site":/usr/share/nginx/html sample-site
curl localhost:8080        # Sample v2 —— 无 build 即变
docker rm -f sample        # 测试完毕，即弃
```



一张表带走全部要点

方面	要点
测试诉求	隔离、可复现、**测完即弃**
初始 Dockerfile	`nginx:alpine` + `COPY site/` 进根目录
两种宿主	Nginx(`/`) vs Tomcat(`/sample/`)
测试动作	run -p → curl → logs/exec 佐证
改网站	**bind mount**即时 · build 交付 · exec 应急
综合	Dockerfile/build/run/exec/logs/mount 全打通

静态站测明白了, 下一步给网站装个"Java 引擎"

> 下一步: 案例二 —— 基于 Docker 的 Java Web 应用, 把这个壳替换成真正的 war 工程

谢谢大家

Q & A